

Machine Learning for Physics

Introduction

Filippo Masi[★]

Laboratoire Jean Kuntzmann (LJK), TRIPOP team

Université Grenoble Alpes, Inria, MIAI | Artificial Intelligence & Mechanics (AIM)



[★]filippo.masi@inria.fr

Outline

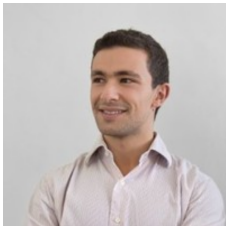
- 1 Overview of the course
- 2 Why machine learning for physics?

Overview of the course

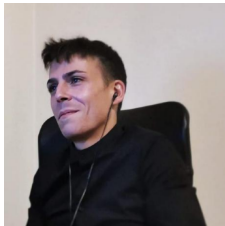
Introduction



Vincent Acary



Michael Arbel



Filippo Masi



Henri Leroy

Introduction

Format

- ▶ 3 ECTS
- ▶ 18 hours
- ▶ Lectures and hands-on sessions
- ▶ Machine learning for the solution and modelling of physical systems

Main themes

- ▶ Physics-informed learning
- ▶ Operator learning
- ▶ Structured and differentiable learning
- ▶ Comparison with numerical solvers

Central question:

How can machine learning complement numerical methods for modelling and solving physical problems?

Prerequisites

Prerequisites

- ▶ Probability — Master 1 level
- ▶ Statistics — Master 1 level

Useful background

- ▶ Linear algebra and multivariable calculus
- ▶ Numerical methods and optimization
- ▶ Ordinary and partial differential equations
- ▶ Python programming
- ▶ Machine learning/artificial neural networks

Structure of the course

1. Physics-informed learning

- ▶ Neural networks as approximations of physical fields.
- ▶ Physics-Informed Neural Networks (PINNs).

2. Operator learning

- ▶ Learn mappings between functions rather than individual solutions.
- ▶ Fourier Neural Operators (FNO), and related architectures.

3. Differentiable solvers and learning

- ▶ Differentiable solvers.
- ▶ Implicit and equilibrium models.

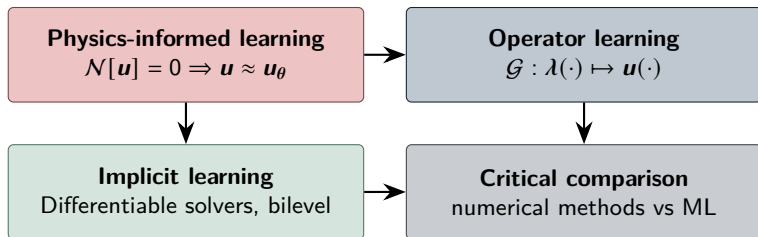
4. ML versus classical numerical methods

- ▶ Numerical methods versus learning-based approaches.
- ▶ Accuracy, computational cost, scalability, and data requirements.

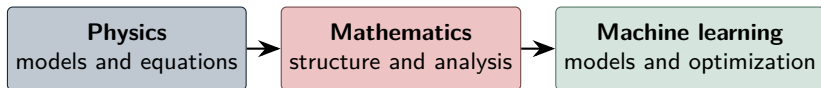
Objectives of the course

By the end of this unit, you should be able to:

- ▶ formulate learning problems for the modelling and solution of physical systems;
- ▶ incorporate mathematical and physical structure into machine learning models;
- ▶ distinguish between solution learning, operator learning, and implicit/differentiable solvers;
- ▶ assess when a learning-based method is appropriate and when a classical numerical solver is preferable;
- ▶ benchmark ML approaches against FEM/FDM/FVM in terms of accuracy, computational cost, scalability, data requirements, and physical consistency.



Objectives of the course



- ▶ Start from a physical or numerical problem, not from an architecture.
- ▶ Identify what is known: equations, constraints, symmetries, data.
- ▶ Decide what should actually be learned.
- ▶ Compare against an appropriate numerical baseline.
- ▶ Analyze both successes *and* failure modes.

What can we learn from data?

Consider a physical problem written abstractly as

$$\mathcal{N}(\boldsymbol{u}; \boldsymbol{\lambda}) = 0$$

- $\mathcal{N}$: arbitrary differential equation/residual
- $\boldsymbol{u} \in \mathcal{U} \subset \mathcal{R}^{d_U}$: physical variable
- $\boldsymbol{\lambda} \in \mathcal{L} \subset \mathcal{R}^{d_\lambda}$: parameters, coefficients, geometry, initial/boundary conditions

A solution

An operator

Unknown parameters

$$\boldsymbol{u}_\theta(\boldsymbol{x}) \approx \boldsymbol{u}(\boldsymbol{x})$$

$$\mathcal{G}_\theta(\boldsymbol{\lambda}) \approx \boldsymbol{u}$$

$$\boldsymbol{\lambda}_\theta \approx \boldsymbol{\lambda}^\star$$

One physical instance

A family of problems

Inverse problems

The appropriate ML formulation depends on what is known, what is observed, and what must be predicted.

Teaching material and evaluation

Format

- ▶ Lectures: concepts, mathematical formulation, and analysis.
- ▶ Examples: physical problems and numerical benchmarks.
- ▶ Hands-on sessions: implementation and experiments.

Material

Slides, exercises, and code available on the course GitHub repository.

url: <https://github.com/aimechanics/machine-learning-for-physics>

Evaluation

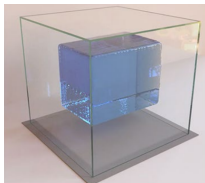
- ▶ Project carried out in small groups.
- ▶ Written report and reproducible code (**40%** of the final mark).
- ▶ Oral defense and individual questions (**60%** of the final mark).

Why machine learning for physics?

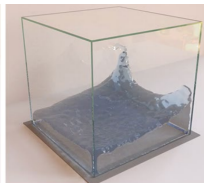
Why machine learning for physics?



Airbus Industrie A321neo D-AVXA



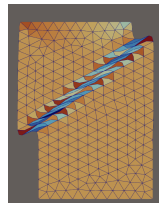
Sanchez-Gonzalez et al. ICML 2020



Boston Dynamics



Badias et al. Int. J. Numer. Methods Eng, 2020



Fabbrini et al. 2026, ALERT

Two questions

1. How can we use **known physics** to improve learning?
2. How can we use **learning** to identify, refine, or discover physics?

Scientific machine learning: at the interface

Traditional scientific computation

Strong physics, little or no data:

- ▶ Finite difference/element methods
- ▶ calibrated physical models.

Data-driven learning

Strong data, weaker explicit physical structure:

- ▶ regression / artificial neural networks
- ▶ reduced and operator models.

Scientific machine learning: at the interface

Traditional scientific computation

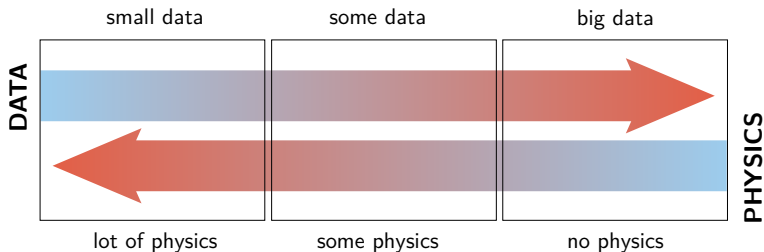
Strong physics, little or no data:

- ▶ Finite difference/element methods
- ▶ calibrated physical models.

Data-driven learning

Strong data, weaker explicit physical structure:

- ▶ regression / artificial neural networks
- ▶ reduced and operator models.



Scientific machine learning: at the interface

Traditional scientific computation

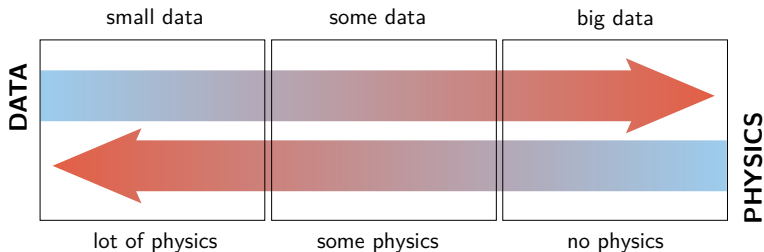
Strong physics, little or no data:

- ▶ Finite difference/element methods
- ▶ calibrated physical models.

Data-driven learning

Strong data, weaker explicit physical structure:

- ▶ regression / artificial neural networks
- ▶ reduced and operator models.



Physics-informed learning

Best of the two worlds: (i) learn from observations while (ii) respecting physics.

Two complementary directions

Physics → machine learning

Embed or enforce what we already know:

- ▶ governing equations
- ▶ conservation laws
- ▶ symmetries and invariances
- ▶ boundary / initial conditions
- ▶ admissibility and constitutive restrictions.

Machine learning → physics

Use data to infer what is missing:

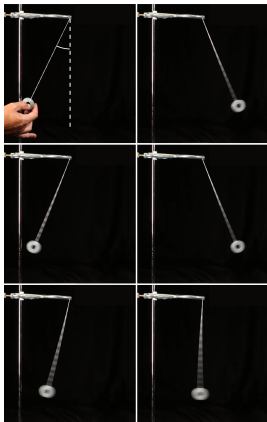
- ▶ physical parameters
- ▶ hidden fields
- ▶ closure / constitutive terms
- ▶ low-dimensional coordinates
- ▶ governing equations or model structure.

Scientific ML is the interaction between prior knowledge and observations.

Example: machine learning

Video of an oscillating pendulum: high-dimensional observation $\mathbf{x}(t_k) = \mathbf{x}_k \in \mathcal{R}^n$

Machine-learning question: compress the measured signal into a small latent state $\mathbf{z}(t)$ and reconstruct the observations?

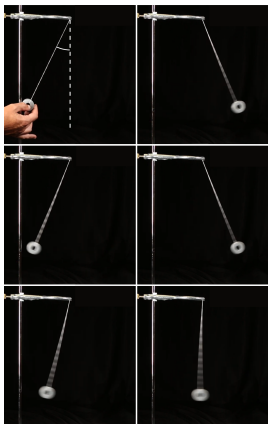


An autoencoder may discover a low-dimensional latent representation that reconstructs the frames but has no reason to correspond to physically meaningful coordinates.

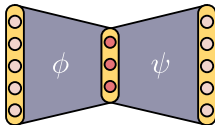
Example: machine learning

Video of an oscillating pendulum: high-dimensional observation $\mathbf{x}(t_k) = \mathbf{x}_k \in \mathcal{R}^n$

Machine-learning question: compress the measured signal into a small latent state $\mathbf{z}(t)$ and reconstruct the observations?



$$\mathbf{z}_k = \phi(\mathbf{x}_k), \quad \tilde{\mathbf{x}}_k = \psi(\mathbf{z}_k)$$

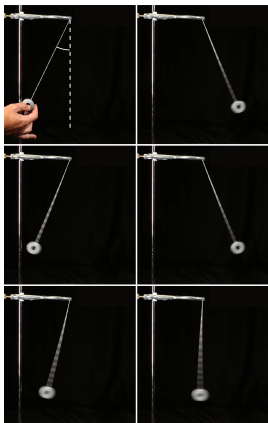


An autoencoder may discover a low-dimensional latent representation that reconstructs the frames but has no reason to correspond to physically meaningful coordinates.

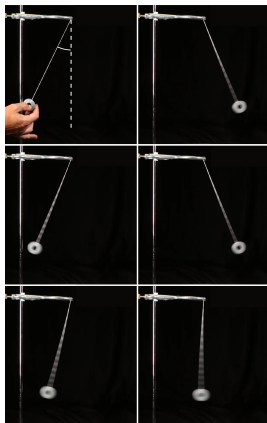
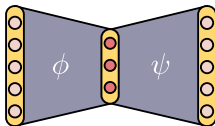
Example: machine learning

Video of an oscillating pendulum: high-dimensional observation $\mathbf{x}(t_k) = \mathbf{x}_k \in \mathcal{R}^n$

Machine-learning question: compress the measured signal into a small latent state $\mathbf{z}(t)$ and reconstruct the observations?



$$\mathbf{z}_k = \phi(\mathbf{x}_k), \quad \tilde{\mathbf{x}}_k = \psi(\mathbf{z}_k)$$

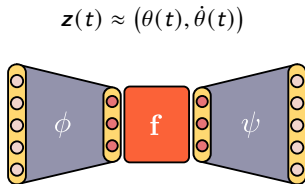
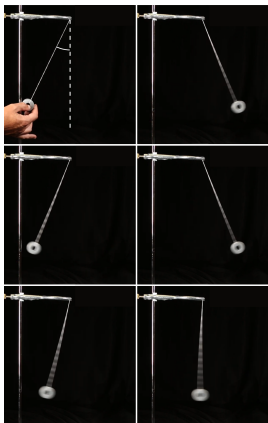


An autoencoder may discover a low-dimensional latent representation that reconstructs the frames but has no reason to correspond to physically meaningful coordinates.

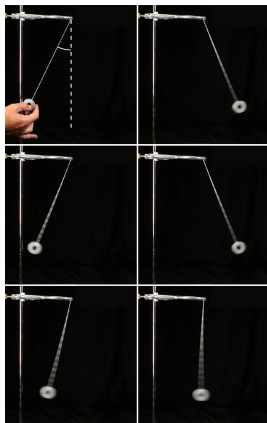
Example: machine learning for physics

Video of an oscillating pendulum: high-dimensional observation $\mathbf{x}(t_k) = \mathbf{x}_k \in \mathcal{R}^n$

Physics-informed machine-learning question: learn physically meaningful coordinates $\mathbf{z}(t)$ and an evolution law $\frac{d\mathbf{z}}{dt}(t) := \dot{\mathbf{z}} = \mathbf{f}(\mathbf{z}, \lambda)$?



$$\ddot{\theta} = -\frac{g}{L} \sin \theta - \delta \dot{\theta}$$



Choosing the state variables or coordinates is already a physics-informed decision.

The physical problem comes first

We start from a physical field $\mathbf{u}$ governed by

$$\mathcal{N}[\mathbf{u}; \lambda] = 0 \quad \text{in } \Omega \times (0, T],$$

with boundary and initial information

$$\mathcal{B}[\mathbf{u}] = 0, \quad \mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}).$$

State

Physical parameters

$$\mathbf{u} = \mathbf{u}(\mathbf{x}, t)$$

λ

temperature, displacement, velocity,
pressure, concentration, phase field, ...

diffusivity, stiffness, viscosity, reaction rates,
wave speed, loads, ...

Before choosing a network: what is known, and what is unknown?

Example: heat conduction and the classical baseline

For a homogeneous material,

$$\rho c \frac{\partial T}{\partial t} - \nabla \cdot (k \nabla T) = q.$$

A forward simulation computes

$$(\Omega, k, \rho, c, q, T_0, \text{BCs}) \mapsto T(\mathbf{x}, t)$$

Classical numerical methods

Finite-difference method (FDM), finite-volume method (FDM), and finite-element method (FEM) discretize the mathematical model and compute an approximation of the state.

Baseline principle

If all inputs are known and only **one accurate solution** of a well-posed problem is required, a mature numerical solver is usually the natural choice.

Why introduce learning around a physical solver?

Repeated queries

One solve may be affordable; 10^3 – 10^6 solves may not be.

Optimization, uncertainty, control, Bayesian inference, digital twins.

Inverse problems

The state is observed only partially, while parameters, fields, sources, or initial conditions are unknown.

Incomplete physics

Balance laws may be known while closures, constitutive relations, or unresolved terms are not.

High-dimensional problems

Stochastic PDEs, kinetic equations, many-body systems, and large parameterized state spaces.

The difficulty is often not the forward PDE, but the task surrounding it.

Repeated queries: learn a map, not just one solution

A repeated-query problem asks for $\lambda \mapsto \mathbf{u}(\lambda)$, or, more generally, a solution operator $\mathcal{G} : \mathbf{a}(\mathbf{x}, t) \mapsto \mathbf{u}(\mathbf{x}, t)$.

Why this can be useful

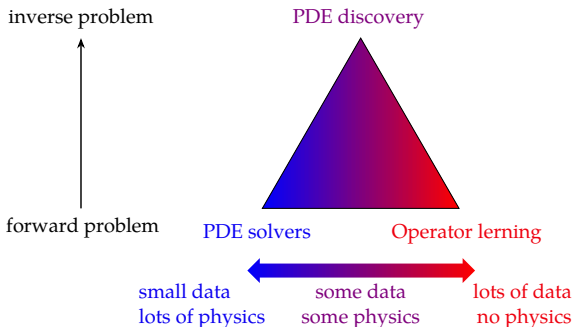
After an expensive offline stage, a learned surrogate may provide fast online evaluations for new inputs. But the offline data-generation cost must be included in the comparison.

Repeated queries: learn a map, not just one solution

A repeated-query problem asks for $\lambda \mapsto \mathbf{u}(\lambda)$, or, more generally, a solution operator $\mathcal{G} : \mathbf{a}(\mathbf{x}, t) \mapsto \mathbf{u}(\mathbf{x}, t)$.

Why this can be useful

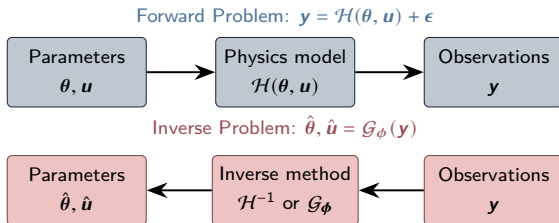
After an expensive offline stage, a learned surrogate may provide fast online evaluations for new inputs. But the offline data-generation cost must be included in the comparison.



N. Boullé, A. Townsend (2024). A mathematical guide to operator learning. In Handbook of Numerical Analysis (Vol. 25, pp. 83-125).

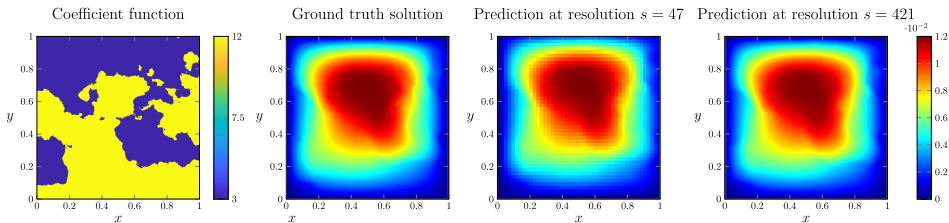
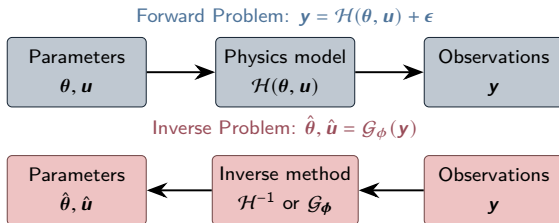
Inverse problems: observations and physics constrain different directions

An inverse problem starts from observations $\mathbf{y}_i = \mathcal{H}_i[\mathbf{u}] + \epsilon_i$ and attempts to infer an unknown state, parameter, field, source, or model component.



Inverse problems: observations and physics constrain different directions

An inverse problem starts from observations $\mathbf{y}_i = \mathcal{H}_i[\mathbf{u}] + \epsilon_i$ and attempts to infer an unknown state, parameter, field, source, or model component.



2D Darcy flow equation $-\text{div}(\mathbf{a}(\mathbf{x})\nabla\mathbf{u})$: super-resolution and inverse problem with operator learning.

N. Boullé, A. Townsend (2024). In Handbook of Numerical Analysis (Vol. 25, pp. 83-125).

Incomplete physics: learn only what is missing

Even when the differential equations are known, important terms may be uncertain:

- ▶ turbulence or subgrid closures
- ▶ material models at unresolved scales
- ▶ unknown sources
- ▶ unknown instabilities
- ▶ spatial/temporal dependency
- ▶ model discrepancy between simulations and experiments.

Incomplete physics: learn only what is missing

Even when the differential equations are known, important terms may be uncertain:

- ▶ turbulence or subgrid closures
- ▶ material models at unresolved scales
- ▶ unknown sources
- ▶ unknown instabilities
- ▶ spatial/temporal dependency
- ▶ model discrepancy between simulations and experiments.

Hybrid modeling: physics + learning

Retain the physical backbone and learn only the missing term:

$$\mathcal{N}_{known}[\mathbf{u}; \lambda] + C_{\theta}[\mathbf{u}] = 0,$$

where $\mathcal{N}_{known}[\mathbf{u}; \lambda]$ is the (partially-) known physical structure, λ is the (un)known parameter set, $C_{\theta}[\mathbf{u}]$ residual term parametrized by unknown θ parameters.

Incomplete physics: learn only what is missing

Even when the differential equations are known, important terms may be uncertain:

- ▶ turbulence or subgrid closures
- ▶ material models at unresolved scales
- ▶ unknown sources
- ▶ unknown instabilities
- ▶ spatial/temporal dependency
- ▶ model discrepancy between simulations and experiments.

Hybrid modeling: physics + learning

Retain the physical backbone and learn only the missing term:

$$\mathcal{N}_{known}[\mathbf{u}; \lambda] + C_{\theta}[\mathbf{u}] = 0,$$

where $\mathcal{N}_{known}[\mathbf{u}; \lambda]$ is the (partially-) known physical structure, λ is the (un)known parameter set, $C_{\theta}[\mathbf{u}]$ residual term parametrized by unknown θ parameters.

A paradigm

The most effective model may lie between the two extremes:

reality is made of what is known and what is ignored.

Why not simply supervised machine learning?

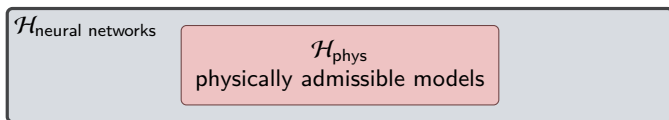
A conventional supervised model fits examples:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \|\mathbf{f}_{\theta}(\mathbf{x}_i) - \mathbf{y}_i\|^2.$$

- ▶ Labels: full-field labels may be expensive, sparse, noisy, indirect, or unavailable.
- ▶ Admissibility: a generic network does not automatically know $\nabla \cdot \mathbf{v} = 0$, $\mathbf{u} \geq 0$, etc.
- ▶ Extrapolation: scientific deployment often asks questions outside the observed training regime.

A neural model searches in a hypothesis class $\mathcal{H} = \{\mathbf{u}_{\theta} : \theta \in \Theta\}$, but physical knowledge can enforce a smaller admissible family

$$\mathcal{H}_{phys} = \{\mathbf{u} \in \mathcal{H} : \mathcal{N}[\mathbf{u}] = 0, \mathcal{B}[\mathbf{u}] = 0, \mathcal{C}[\mathbf{u}] = 0\}.$$

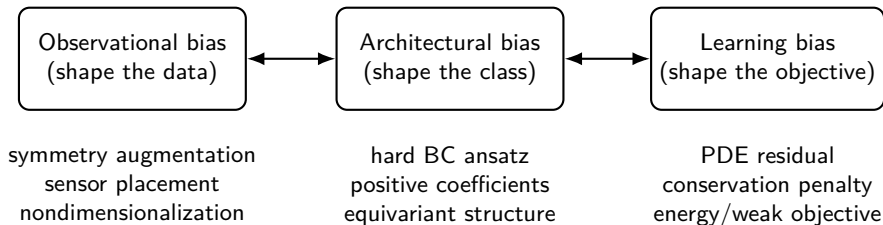


Why physics can improve data efficiency

The constraints reduce the effective search space.

Fewer observations may be needed to select a useful model within the admissible family.

Three mechanisms for inserting physical knowledge



Observational bias

Physics shapes what is measured, sampled, augmented, or nondimensionalized.

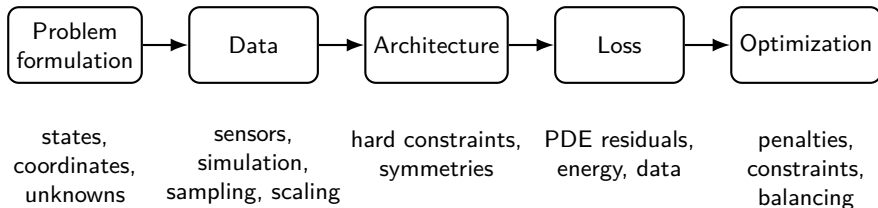
Architectural bias

Physics shapes the trial class through hard constraints, positivity, invariance, equivariance.

Learning bias

Physics shapes the objective through residuals, penalties, weak forms, or energy functionals.

Five opportunities to inject physics



The important point

Physical knowledge can alter and structure every stage of the learning workflow.

Physics in the problem formulation and in the data

1. Problem formulation

Before choosing a network, decide what is represented:

- ▶ Cartesian or intrinsic coordinates?
- ▶ displacement, angular displacement?
- ▶ scalar coefficient or spatial field?
- ▶ full state or reduced coordinates?

3. Architecture

Make properties true by construction:

$$u_{\theta}(x) = x(1-x)N_{\theta}(x)$$

for $u(0) = u(1) = 0$.

Positivity, equivariance,
Hamiltonian structure,
divergence-free fields, etc.

2. Data collection and curation

Physics can guide

- ▶ sensor placement and observability;
- ▶ oversampling near interfaces or fronts;
- ▶ symmetry-based augmentation;
- ▶ multi-fidelity data;
- ▶ nondimensionalization and scaling.

4. Loss

Penalize violations:

$$\mathcal{L} = w_d \mathcal{L}_d + w_r \mathcal{L}_r + w_b \mathcal{L}_b.$$

Residuals, data, BCs,
conservation, energy, weak
objectives.

5. Optimization

Soft penalties are not exact constraints.

Physics can motivate
constrained optimization,
augmented Lagrangians,
adaptive balancing, etc.

Principled workflow



A physical benchmark is especially valuable

When the governing model and reference solution are understood, we can study *why* an architecture, loss, or optimizer succeeds or fails instead of only checking whether it produces a plausible prediction.

What problem are we solving?

Different unknowns imply fundamentally different learning problems.

Forward problem

Known: $\mathcal{N}$, λ , BCs, ICs

Unknown: $\mathbf{u}$.

Inverse parameter problem

Partial observations of $\mathbf{u}$ are available. Infer one or more unknown parameters λ .

1. What is known?

Complete model? Unknown coefficients? Sparse observations? Symmetries?
Conservation laws? Trusted simulator?

2. What is unknown?

$\mathbf{u}$, λ , $\lambda(\mathbf{x})$, $\mathcal{N}$ or part of $\mathcal{N}$.

3. What will the model be used for?

One accurate solve? Repeated queries? Real-time prediction? Inverse identification?
Control? Uncertainty propagation?

Field identification

Infer an unknown function $\lambda = \lambda(\mathbf{x})$. For example: conductivity or spatial stiffness.

Hybrid / discovery problem

Infer a missing component $\mathcal{C}$ or, more ambitiously, part of the governing model $\mathcal{N}$ itself.

Summary

1. Physics-informed learning (F. Masi)

- ▶ Neural networks as approximations of physical fields and physics-informed machine learning.

2. Operator learning (H. Leroy)

- ▶ Learn mappings between functions rather than individual solutions: neural operators.

3. Differentiable solvers and learning (M. Arbel)

- ▶ Differentiable solvers, implicit and equilibrium models.

Material

Slides, exercises, and code available on the course GitHub repository.

Evaluation

- ▶ Project carried out in small groups (selection of the projects at session #4).
- ▶ Written report and reproducible code (**40%** of the final mark).
- ▶ Oral defense and individual questions (**60%** of the final mark).